

DMA ÷ Dynamic Memory Allocation

SMA (Static Memory Allocation)

Variable made using declaration statement

- Memory allocated during compile time is called static memory
- The memory allocated is fixed and cannot be increased or decreased during run time

```
int x;  
float y;  
int *p;  
struct date d1;
```

Drawbacks ÷
fixed memory
Non-flexible
Wastage of Memory

DMA

Variable made by calling function

The process of allocating memory at the time of execution is called dynamic memory allocation.

functions such as :

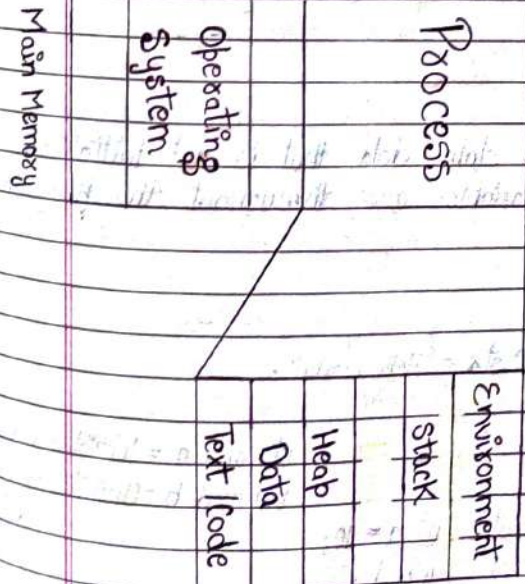
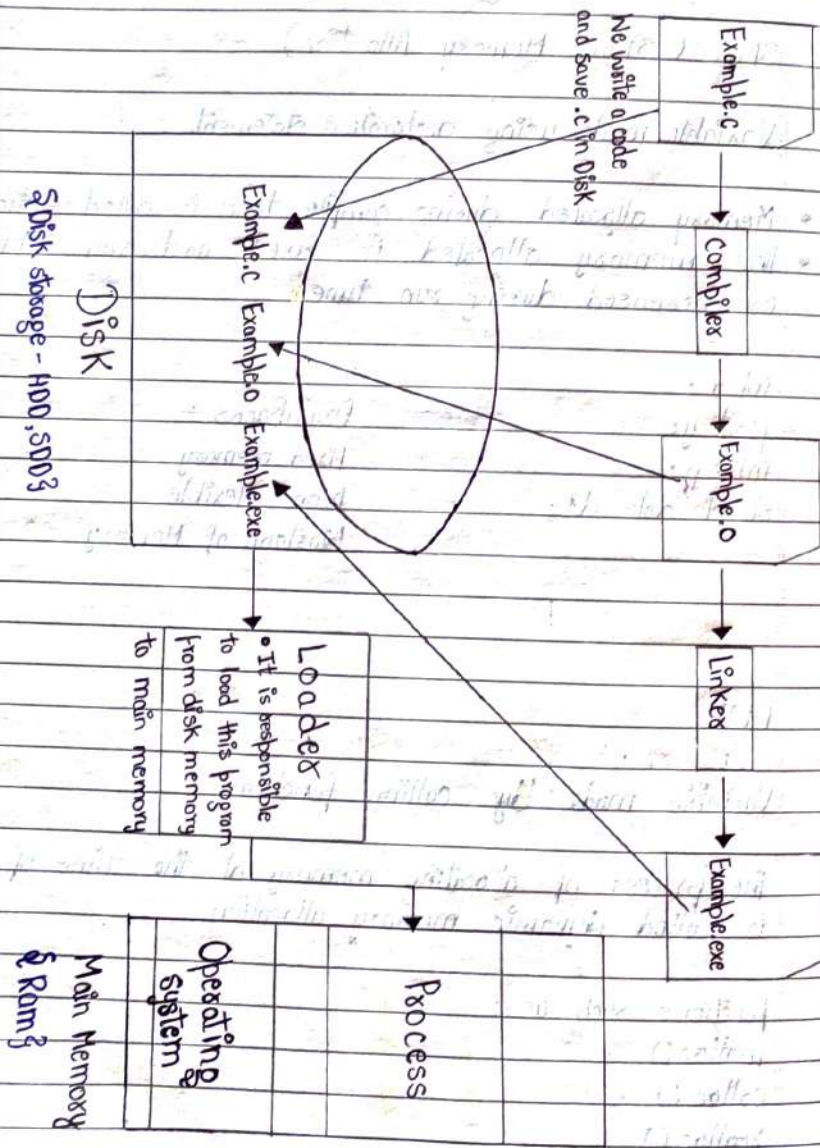
malloc()

calloc()

realloc()

free()

Memory segments of C Program



Text / Code segment

- It contains program instruction for execution
- It is read only section of program and cannot be changed during execution.
- It is placed in lower memory address so its not overwritten by heap or stack.

Data segment → Uninitialized data
 Initialized data

Initialized data

It contains global & static variables that are explicitly initialized.

Lifetime of variables in this section is throughout the execution of program.

Variables in this section can be changed.

Uninitialized data

It contains global and static data that is not initialized. Lifetime of these variables are throughout the program execution.

Example Program :-

```
const char *str = "Hello world";
```

```
int val;
```

```
int main()
```

2

```
static int a = 10;
```

```
static int b;
```

```
return 0;
```

2.

str and a = Initialized data
val and b = Uninitialized data

Heap segment

: 449

Heap is the segment of memory where dynamic memory allocation takes place.

Heap memory must be managed by the program statements.

malloc, calloc, realloc, free are the functions used to manage heap memory.

If memory not managed properly then it will lead to memory leak problems.

Stack segment

Memory block created that holds ~~that holds~~ the information of the functions known as stack frame or activation records.

Local variable also get memory in stack frame.

Stack frame is popped / jump (remove) from stack when function returns.

Environment

Holds Command line Arguments & Environment variables.

DMA :

• malloc ()

malloc is a built-in function declared in the header file `<stdlib.h>`.

malloc is the short name for "memory allocation" and is used to dynamically allocate a single large block of contiguous memory according to the size specified.

Syntax ÷

malloc (8)

Heap segment

- It will reserved one complete block in Heap of 8 bytes

8 bytes

- And it will return base address of the block

2000

- So to store the address we suppose to take a pointer

void* malloc (size_t size)

void pointer

↳ unsigned int

void pointer only because malloc doesn't know which type of value we are going to store.

So according to our need we can type cast it.

`void * malloc (size_t size)`

↳ It is not a good practice

* `int * ptr;`
`ptr = (int *) malloc (n * size of (int));`

↓
no. of integer
we want to store

↳ It is a good practice
because int has different
range in different compilers
So it is good to let
compiler decode the size

As we know malloc returns
void (pointer) so we need to
type-cast it because our
pointer is defined as integer
pointer
which is holding the
address of that memory
block.

* As we know that
malloc return address so
its return type will be

- If we not initialised memory allocated by malloc it by default contains garbage value
- If there is not sufficient memory available in heap then malloc() returns null (NULL).

So we should always check the value returned by malloc().

```
int * ptr;
ptr = (int *) malloc (n * size of (int));
if (ptr == NULL)
    printf("Sufficient memory not available");
```



```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int n, i, *ptr;
```

```
    printf("Enter total no. of values:");
```

```
    scanf("%d", &n);
```

```
    ptr = (int*) malloc (n * size of (int));
```

```
    printf("Enter values:");
```

```
    for (i=0; i<n; i++)
```

```
    {
```

```
        scanf("%d", (ptr+i));
```

```
    }
```

```
    printf("The entered values are:");
```

```
    for (i=0; i<n; i++)
```

```
    {
```

```
        printf("%d", *(ptr+i));
```

```
    }
```


Calloc () in C

Contiguous allocation

Built in function in stdlib.h

Used to dynamically allocate multiple blocks of memory & each block is of same size.

The calloc () function is used to allocate multiple blocks of memory. It is similar to malloc () function except for two differences. The first one:

It takes two arguments

`void * calloc (size_t n, size_t size)`

↓
no. of blocks

↓
size of each block

`ptr = (int *) calloc (5, size of (int));`

← 4 bytes →

`ptr = (int *) malloc (5 * size of (int));`

← 20 bytes →

The other difference b/w calloc and malloc () is that the memory allocated by malloc () contains garbage value while the memory allocated by calloc is initialized to zero.

If there is not sufficient memory available in heap then calloc () returns NULL.

Realloc()

It is use to increase or decrease the memory allocated by malloc() or calloc().

The function realloc() is used to change the size of the memory block.

- * It alters the size of the memory block without losing the old data.

This function takes two arguments, first is a pointer to the block of memory that was previously allocated by malloc() or calloc() and second one is the new size for that block.

`void * realloc (void * pointer, size_t size)`

`int * ptr;`

`ptr = (int *) malloc (5 * sizeof(int));`

1	2	3	4	5
---	---	---	---	---

1000

`ptr = (int *) realloc (ptr, 7 * sizeof(int));`

If possible realloc will extend the old memory block in heap.

1000

1
2
3
4
5
6
7

Extended block in
same old memory block

If not possible then `realloc` will allocate new memory block in heap, copy the previous content and deallocate the previous memory block.

And return the new address of new allocated memory.

2000

1

2

3

4

5

6

7

1000

1

2

3

4

5

On failure, `realloc()` returns `NULL` and in this case the old memory is not deallocated and it remains unchanged.


```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main ( )
```

```
{
```

```
    int *ptr, n, i;
```

```
    printf ("Enter value of n:");
```

```
    scanf ("%d", &n);
```

```
    ptr = (int*) calloc (n, size of (int));
```

```
    printf ("Enter values:");
```

```
    for (i=0; i<n; i++)
```

```
    {
```

```
        scanf ("%d", (ptr+i));
```

```
    }
```

```
    printf ("Enter updated n:");
```

```
    scanf ("%d", &n);
```

```
    int *ptr1 = (int*) realloc (ptr, n * sizeof(int));
```

```
    printf ("Previous address = %d, new address = %d", ptr, ptr1);
```

```
    printf ("Values are:");
```

```
    for (i=0; i<n; i++)
```

```
    {
```

```
        printf ("%d", *(ptr+i));
```

```
    }
```

```
    free (ptr1);
```

```
}
```


Date _____
Page _____

free()

release the dynamically allocate memory.
built-in function declared in `stdlib.h`

`void free (pointer name)`

→ As it is a function its return type is void. It will return nothing.

The dynamically allocated memory is not automatically released: it will exist till the end of program. If we have finished working with the memory allocated dynamically, it is our responsibility to release that memory so that it can be reused.

The function `free()` is used to release the memory space allocated dynamically.

The memory released by `free()` is made available to the heap again and can be used for some other purpose.

```
int * ptr;  
ptr = (int *) malloc (n * sizeof(int));  
free(ptr);
```


int * ptr;

(int *) malloc (2 * sizeof(int))

Heap segment

1000	10
	20

- free function doesn't erase the previous data.

- After freeing if we try to access data it will show undefined-behaviour.

ptr

1000

- After using free function the pointer (ptr) will act as Dangling pointer

int * ptr;
(int *) malloc (2 * sizeof(int))
free(ptr)

Memory leak in C :

Due to improper use of heap segment

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main ()
```

```
{
```

```
    int ch = 1;
```

```
    int * ptr;
```

```
    while (ch < 50)
```

```
    {
```

```
        printf("memory leak demo\n");
```

```
        ptr = (int *) malloc (4000 * sizeof(int));
```

```
        printf("continue ?? press 1 for continue");
```

```
        scanf("%d", &ch);
```

```
    }
```

```
}
```

Here we have not freed the memory allocated in heap segment

In every loop iterate we are allocating memory for 4000 int in heap segment and we are not freeing it.

So the memory allocated in heap segment will act like garbage after every iteration and which will cause memory leak

The consequence of the memory leak is that it reduces the performance of the computer by reducing the amount of available memory.

The system slows down vastly.